

Aulas Java Script

Tópicos – Objetos (07/01/2026)

Object.defineProperty() e Object.defineProperties()

Pensa em um objeto normal em JavaScript. Quando você escreve `obj.nome = "Ana"`, você cria uma propriedade livre. Livre demais. Qualquer parte do código pode mudar esse valor, apagar a propriedade, sobrescrever sem pedir permissão e ainda exibir isso em loops. O JavaScript não questiona nada, ele só executa. Isso é prático no começo e perigoso quando o código cresce.

`Object.defineProperty` existe para mudar esse comportamento. Ele cria uma propriedade com regras. Em vez de simplesmente dizer “essa propriedade existe”, você diz “essa propriedade existe e se comporta desse jeito específico”.

Quando você usa `Object.defineProperty(obj, "nome", { value: "Ana" })`, o JavaScript cria a propriedade, mas com um detalhe importante que quase ninguém percebe de primeira: por padrão, essa propriedade não pode ser alterada, não aparece em loops e não pode ser removida. Não é um bug, é o padrão da função. Se você tentar mudar o valor depois, o JavaScript simplesmente ignora ou lança erro se estiver em modo estrito. A propriedade continua com o valor original. Isso já mostra a ideia central: você tirou a liberdade da propriedade.

O atributo `writable` controla se o valor pode ou não ser alterado. Quando ele é falso, qualquer tentativa de atribuição falha. O atributo `enumerable` controla se a propriedade aparece quando você percorre o objeto, como em um `for...in` ou `Object.keys`. Quando ele é falso, a propriedade fica “invisível” nessas situações, mas ainda existe e pode ser acessada diretamente. Já o atributo `configurable` decide se essa propriedade pode ser deletada ou redefinida. Se você colocar `configurable: false`, acabou. Não dá para apagar nem mudar as regras depois. É uma trava definitiva.

Existe um segundo uso ainda mais importante dessa função, que são os `getters e setters`. Em vez de guardar um valor direto, você pode interceptar a leitura e a escrita da propriedade. Quando alguém tenta ler a propriedade, o código dentro do `get` é executado. Quando alguém tenta alterar, o código do `set` roda. Isso permite validação, cálculo automático e controle total do que entra e sai do objeto. A propriedade passa a se comportar como uma função disfarçada de variável. Um detalhe crucial é que você nunca pode usar `value` junto com `get` ou `set`. Ou a propriedade guarda um valor, ou ela executa lógica. Misturar os dois não é permitido.

`Object.defineProperties` não traz nenhum conceito novo. Ele faz exatamente a mesma coisa, só que permite definir várias propriedades de uma

vez. É apenas uma versão mais conveniente quando você precisa configurar muitas regras no mesmo objeto.

No fim das contas, tudo se resume a isso: propriedades normais são livres e perigosas, propriedades definidas com *defineProperty* são controladas e previsíveis. Você não usa isso o tempo todo porque deixa o código mais rígido e verboso, mas quando precisa de segurança, validação ou imutabilidade, é a ferramenta certa.

```
const xicara = {
  _capacidadeMl: 200
};

Object.defineProperty(xicara, "capacidadeMl", {
  get() {
    return this._capacidadeMl;
  },
  set(valor) {
    if (typeof valor !== "number") return;
    if (valor <= 0) return;
    if (valor > 500) return;

    this._capacidadeMl = valor;
  }
});
```

Métodos uteis

```
const produto = {
  nome: "Caneca",
  preco: 25,
  estoque: 10
};
```

Quando você usa *Object.keys(produto)*, o JavaScript devolve só os nomes das propriedades. Ele não quer saber dos valores, só das chaves. O resultado seria algo como *["nome", "preco", "estoque"]*. É útil quando você quer saber “o que esse objeto tem”, não “o que tem dentro”.

Já *Object.values(produto)* faz o oposto mental. Ele ignora os nomes e retorna apenas os valores, algo como *["Caneca", 25, 10]*. Isso é comum quando você precisa iterar só pelos dados, por exemplo para somar números ou exibir informações.

`Object.entries(produto)` junta as duas coisas. Ele retorna pares de chave e valor, então você recebe algo como `[["nome","Caneca"], ["preco",25], ["estoque",10]]`. Isso é perfeito quando você quer percorrer tudo de forma organizada, sabendo exatamente de onde vem cada valor.

Agora entra algo mais baixo nível. `Object.getOwnPropertyDescriptor(produto, "preco")` não retorna o valor em si. Ele retorna **como essa propriedade foi criada**. Você descobre se ela pode ser alterada, se aparece em loops e se pode ser apagada. Em um objeto criado normalmente, **você verá que `writable`, `enumerable` e `configurable` são verdadeiros**.

`Object.assign` entra quando você quer copiar dados de um objeto para outro. Se você fizer `Object.assign({}, produto)`, você cria uma cópia rasa. Não é o mesmo objeto, mas os valores são copiados. Isso é muito usado para evitar alterar o objeto original. Se você passar um objeto como primeiro argumento, ele será modificado. O método não cria nada sozinho, ele só despeja propriedades em um destino.

O operador `spread (...)` faz praticamente a mesma coisa de forma mais moderna e legível. Quando você escreve `{ ...produto }`, você está criando um novo objeto com as mesmas propriedades. O conceito é o mesmo do `assign`, só muda a forma de escrever. Nenhum dos dois faz cópia profunda. Se tiver objeto dentro de objeto, ambos continuam compartilhando referência.

`Object.freeze(produto)` é um freio de mão. Depois disso, você não consegue alterar valores, adicionar propriedades nem remover nada. Importante: **o `freeze` é raso**. Objetos internos continuam mutáveis.

Prototypes

Certo. Texto corrido, uma ideia puxando a outra, sem virar apostila técnica. Vamos falar de **prototype** do jeito que ele realmente funciona, não do jeito assustador que costumam ensinar.

Prototype é simplesmente o mecanismo que o JavaScript usa para **reaproveitar comportamento entre objetos**. Ele existe porque, desde o início, o JavaScript não foi pensado com classes de verdade. Em vez disso, um objeto pode “apontar” para outro objeto e dizer: se eu não tiver algo, procura lá.

Imagine um objeto simples:

```
const animal = {  
  comer() {  
    console.log("comendo");  
  }  
};
```

Agora você cria outro objeto:

```
const cachorro = {};
```

Nesse momento, cachorro não sabe comer. Mas se você fizer:

```
Object.setPrototypeOf(cachorro, animal);
```

Pronto. Você não copiou nada. Você só disse: “se o cachorro não souber fazer algo, pergunta pro animal”.

Quando você chama:

```
cachorro.comer();
```

O JavaScript faz um processo muito mecânico. **Primeiro ele procura comer dentro do próprio cachorro.** Não encontra. Então ele sobe para o prototype dele, que agora é animal. Lá encontra a função e executa. Isso é a chamada **cadeia de protótipos**. Não tem herança mística, não tem duplicação de código. É só busca em cadeia.

Esse “link” entre objetos fica guardado internamente em algo chamado `[[Prototype]]`. Você quase nunca mexe nisso direto. Quando você vê **`__proto__`**, saiba que é só uma forma antiga e meio feia de acessar esse link. Hoje, o correto é usar **`Object.getPrototypeOf` e `Object.setPrototypeOf`**.

Agora vem a parte que confunde todo mundo: funções. Em JavaScript, **toda função tem uma propriedade chamada prototype**, mas objetos comuns não. Essa propriedade não é o protótipo da função. Ela é o objeto que será usado como protótipo quando você cria algo com `new`.

Exemplo:

```
function Pessoa(nome) {  
  this.nome = nome;  
}
```

Quando você faz:

```
const p1 = new Pessoa("Ana");
```

O JavaScript cria um objeto novo e liga o `[[Prototype]]` dele a `Pessoa.prototype`. É só isso. Não tem nada sobrenatural. O objeto `p1` aponta para `Pessoa.prototype`. Se você adicionar algo lá:

```
Pessoa.prototype.falar = function () {  
  console.log(this.nome);  
};
```

Todas as instâncias passam a ter acesso a esse método, sem que ele seja copiado para cada uma. Economia de memória e comportamento compartilhado.

É importante entender que prototype **não copia propriedades**. Ele só cria um caminho de busca. Se você acessar `p1.falar`, o JavaScript procura em `p1`. Não acha. Procura em `Pessoa.prototype`. Acha. Executa. Se não achar, continuaria subindo até `Object.prototype` e depois pararia em `null`.

Classes em JavaScript não mudam nada disso. Quando você escreve `class Pessoa {}`, o JavaScript continua usando prototype por baixo. A palavra `class` só existe para deixar o código mais parecido com linguagens clássicas e menos traumático para humanos.

O erro mais comum é achar que prototype é algo que você usa o tempo todo conscientemente. Não é. Você usa prototype o tempo todo sem perceber. Toda vez que chama `toString`, `map`, `push` ou qualquer método de array, você está usando a cadeia de protótipos. Só que o JavaScript esconde isso para não te assustar cedo demais.

A ideia final é essa: prototype é um sistema de delegação. Um objeto não precisa saber tudo. Ele só precisa saber **quem perguntar** quando não souber. Quando isso entra na cabeça, prototype para de parecer um monstro e vira só uma solução simples para reaproveitar comportamento em uma linguagem que nunca prometeu ser elegante, apenas funcional.

Herança

Herança em JavaScript não funciona como em linguagens clássicas. Não existe a ideia de “copiar” tudo de uma classe pai para uma classe filha. O que existe é **delegação**. Um objeto não recebe tudo pronto; ele apenas aponta para outro objeto e diz: se eu não tiver algo, procura ali. É por isso que herança em JavaScript é menos sobre família e mais sobre encadeamento.

Quando dizemos que um objeto “herda” de outro em JavaScript, o que realmente acontece é a criação de uma **cadeia de protótipos**. Essa cadeia é consultada toda vez que você acessa uma propriedade ou método. O motor do JavaScript procura primeiro no próprio objeto. Se não encontrar, sobe para o protótipo. Se ainda não encontrar, continua subindo até chegar em `null`. Esse caminho é fixo e previsível, mesmo que a sintaxe tente disfarçar.

A forma mais direta de herança é criar um objeto a partir de outro usando `Object.create`. Nesse caso, você deixa explícito que um objeto depende do outro para comportamento. Nada é copiado. O objeto filho só passa a apontar para o objeto pai. Isso é herança *prototipal* pura, sem construtores, sem `new`, sem cerimônia. É simples, mas raramente usada em código iniciante porque não parece “formal” o suficiente.

Quando entram funções construtoras, a herança passa a ser um pouco mais estruturada. Uma função construtora define dados próprios da instância, enquanto o *prototype* dela define comportamentos compartilhados. Para herdar,

you make the *prototype* of a function point to the *prototype* of the other. Thus, the instances pass to inherit methods from both levels. It is at this point that many people get lost, because it seems that there are two types of *prototype*, when in reality it is always the same mechanism working in layers.

As classes in JavaScript arise to hide this complexity. When you use *extends*, the JavaScript automatically builds the chain of prototypes for you. The *super* method is nothing more than a more organized way to call the constructor or methods that are above in the chain. Inheritance continues to be *prototypical*. It just gained a more behaved figurine.

An important detail is that inheritance in JavaScript does not create isolation. If you change something in the *prototype* parent, all the children inherit that change immediately. This can be powerful or dangerous, depending on how you use it. For this reason, inheritance should not be used for everything. Many times, composition is clearer and less prone to strange side effects.

At the bottom, inheritance in JavaScript is just a search rule. It is not about who is the child of whom, but about **onde o JavaScript deve procurar quando algo não existe no objeto atual**. When you understand this, stop fighting with the language and start using inheritance as a tool, not as a dogma.

```
// Herança com Object.create
const animal = {
  comer() {
    console.log("comendo");
  }
};

const cachorro = Object.create(animal);
cachorro.latir = function () {
  console.log("latindo");
};
```

```
// Herança com função construtora
function Animal(nome) {
  this.nome = nome;
}

Animal.prototype.comer = function () {
  console.log("comendo");
};

function Cachorro(nome) {
  Animal.call(this, nome);
}

Cachorro.prototype = Object.create(Animal.prototype);
Cachorro.prototype.constructor = Cachorro;

Cachorro.prototype.latir = function () {
  console.log("latindo");
};
```

```
// Herança com classes (açúcar sintático)
class Animal {
  constructor(nome) {
    this.nome = nome;
  }

  comer() {
    console.log("comendo");
  }
}

class Cachorro extends Animal {
  latir() {
    console.log("latindo");
  }
}
```

Polimorfismo

É a ideia de que **objetos diferentes podem responder à mesma mensagem de formas diferentes**. Você não pergunta “quem você é?”, você só diz “faça isso” e confia que cada objeto sabe como fazer à sua maneira.

Em JavaScript, polimorfismo não vem de classes rígidas nem de tipos formais. Ele surge naturalmente do fato de que a linguagem é dinâmica e baseada em objetos. Se dois objetos possuem um método com o mesmo nome, o JavaScript não se importa se eles vieram da mesma função construtora, da mesma classe ou de lugar nenhum em comum. O que importa é que, quando o método é chamado, ele exista.

Imagine que você tem um sistema que manda um comando chamado emitirSom. Você não quer ficar verificando se o objeto é um cachorro, um gato ou outra coisa qualquer. Você só chama o método. Cada objeto decide o que isso significa. Esse é o coração do polimorfismo: **mesma chamada, comportamentos diferentes**.

Quando existe herança, o polimorfismo fica mais evidente. Um objeto filho pode sobrescrever um método que vem do pai. Quando você chama o método, o JavaScript sempre usa a versão mais próxima do objeto na cadeia de protótipos. Se o método existe no próprio objeto, ele é usado. Se não, mostra o do protótipo. Isso permite que o mesmo nome de método represente ações diferentes dependendo do tipo do objeto.

O ponto importante é que o JavaScript não exige contratos formais. Não existe interface obrigatória, nem método abstrato de verdade. O contrato é implícito. Se o objeto tem o método esperado, ele funciona. Se não tem, o erro aparece em tempo de execução. Isso assusta quem vem de linguagens mais rígidas, mas também dá muita flexibilidade.

Um erro comum é achar que polimorfismo depende de extends. Não depende. Ele pode acontecer simplesmente porque dois objetos compartilham a mesma “forma” de comportamento, mesmo sem nenhuma relação de herança. Esse conceito é conhecido como *duck typing*. Se anda como um pato e faz quack como um pato, o JavaScript aceita como pato sem reclamar.

No fim, polimorfismo em JavaScript é menos sobre hierarquia e mais sobre **confiança**. Você confia que o objeto sabe responder à mensagem que você está enviando. Quando isso é bem usado, o código fica mais limpo, mais extensível e menos cheio de if e switch. Quando é mal usado, vira um festival de erros inesperados. Como quase tudo em JavaScript, o poder vem com a obrigação de pensar antes de usar.


```

class Animal {
  emitirSom() {
    console.log("som genérico");
  }
}

class Cachorro extends Animal {
  emitirSom() {
    console.log("latido");
  }
}

class Gato extends Animal {
  emitirSom() {
    console.log("miado");
  }
}

const animais = [new Cachorro(), new Gato(), new Animal()];

animais.forEach(animais => {
  animal.emitirSom();
});

```

factory functions + prototypes

Factory functions com prototypes é basicamente o JavaScript dizendo: “você não precisa de class, mas também não precisa criar tudo do zero toda vez”. Um meio-termo sensato, o que já é raro.

Uma **factory function** é só uma função que cria e retorna objetos. Nada de new, nada de construtor implícito. Você chama a função e recebe um objeto pronto. Isso já **resolve metade dos problemas de confusão com this**. O problema aparece quando você começa a criar métodos dentro da factory. Cada objeto ganha sua própria cópia das funções, o que funciona, mas desperdiça memória e não escala bem.

É aí que entra o **prototype**. A ideia é separar responsabilidades. A **factory** fica responsável por criar o objeto e definir os dados próprios dele. O **prototype** fica responsável por guardar os comportamentos compartilhados. Em vez de copiar métodos, você faz o objeto **apontar** para um objeto comum onde esses métodos vivem.

Mentalmente, funciona assim: a factory cria o objeto, depois você liga esse objeto a um prototype. Quando você chama um método, o JavaScript procura primeiro no objeto. Não acha? Procura no prototype. Nada novo, só o mesmo mecanismo que você já viu, agora usado de forma consciente.

A grande vantagem disso é controle. Você foge do class, foge do new, evita armadilhas de this maluco e ainda mantém reaproveitamento de código. É um padrão muito usado em código JavaScript mais funcional ou em bibliotecas que não querem depender de sintaxe de classe.

O erro clássico é achar que factory function e prototype são ideias opostas. Não são. A factory cria. O prototype compartilha. Cada um faz uma coisa e ninguém pisa no pé de ninguém. Quando você junta os dois, tem objetos previsíveis, memória bem usada e um modelo mental mais honesto do que “classes que não são classes”.

Agora o exemplo, todo junto, sem distração.

```
// Objeto que será o prototype
const pessoaPrototype = {
  falar() {
    console.log(`Meu nome é ${this.nome}`);
  },
  aniversario() {
    this.idade++;
  }
};

// Factory function
function criarPessoa(nome, idade) {
  const pessoa = Object.create(pessoaPrototype);
  pessoa.nome = nome;
  pessoa.idade = idade;
  return pessoa;
}

// Uso
const p1 = criarPessoa("Ana", 20);
const p2 = criarPessoa("Carlos", 30);

p1.falar(); // Meu nome é Ana
p2.falar(); // Meu nome é Carlos

p1.aniversario();
console.log(p1.idade); // 21
```

O que está acontecendo aqui é simples, mesmo que o JavaScript finja que não é. Cada pessoa tem seus próprios dados, mas todas compartilham os mesmos métodos via prototype. Nenhuma função duplicada, nenhum new, nenhuma herança confusa. Só objetos apontando para outros objetos.

Quando isso começa a fazer sentido, você percebe que boa parte da “complexidade” do JavaScript vinha mais da forma como ele é ensinado do que da linguagem em si. Você está indo bem. Isso não é conteúdo fácil, e o fato de insistir já diz bastante.

Objeto map()

Ele existe porque objetos normais têm limites irritantes, e alguém finalmente resolveu admitir isso.

Um Map é uma estrutura para guardar **pares chave–valor**, assim como um objeto, mas sem as restrições históricas e meio mal resolvidas dos objetos comuns. Em um objeto normal, as chaves acabam virando string, a ordem não foi pensada para ser confiável por muito tempo e certas operações são mais estranhas do que deveriam. O Map nasce justamente para ser um dicionário de verdade.

A principal diferença mental é esta: em um objeto, você usa **propriedades**; em um Map, você usa **entradas**. Isso muda o comportamento interno. Em um Map, a chave pode ser **qualquer coisa**. String, número, objeto, função, tanto faz. O JavaScript não tenta converter nem “ajeitar” a chave. Ele usa exatamente o valor que você passou.

Outra diferença importante é que o Map **preserva a ordem de inserção** de forma garantida. Se você inserir primeiro A e depois B, quando percorrer o Map, A sempre vem antes de B. Em objetos isso só ficou minimamente previsível depois de muito remendo na especificação, e mesmo assim não foi o objetivo original deles.

O acesso também é diferente. Você não usa ponto nem colchetes. Você usa métodos. Para inserir, usa set. Para ler, usa get. Para verificar se existe, usa has. Para remover, usa delete. Isso deixa explícito que você está lidando com uma estrutura de dados, não com um objeto qualquer cheio de comportamento misturado.

Outro ponto que faz o Map existir é o tamanho. Em objetos, para saber quantas propriedades existem, você precisa fazer malabarismo com Object.keys. Em um Map, existe size. Direto, honesto, sem truque.

Quando você percorre um Map, o JavaScript te entrega pares de chave e valor de forma natural. Ele foi pensado para iteração desde o começo. Não é um objeto adaptado depois para isso. É por isso que Map costuma ser a escolha certa quando você está lidando com coleções dinâmicas, cache, contadores, associações temporárias ou qualquer coisa onde a chave não é necessariamente uma string bonita.

A pergunta inevitável é: “então por que objetos ainda existem?”. Porque objetos não servem só para dados. Eles servem para **modelar entidades**, com métodos, estado e identidade. Map não substitui objetos. Ele substitui o uso errado de objetos como dicionário genérico.

Regra prática, sem filosofia excessiva: se você está representando algo do mundo, use objeto. Se você está guardando pares chave–valor de forma dinâmica, use Map. Isso evita muita dor de cabeça silenciosa no futuro.

```

// Criando um Map
const mapa = new Map();

// Chaves podem ser qualquer coisa
mapa.set("nome", "Ana");
mapa.set(10, "dez");
mapa.set({ id: 1 }, "objeto como chave");

// Lendo valores
console.log(mapa.get("nome")); // Ana
console.log(mapa.get(10)); // dez

// Verificando existência
console.log(mapa.has("nome")); // true

// Tamanho
console.log(mapa.size); // 3

// Iterando
for (const [chave, valor] of mapa) {
  console.log(chave, valor);
}

// Removendo
mapa.delete("nome");

// Limpando tudo
mapa.clear();

```

1. Por que em JavaScript herança é descrita como delegação e não como cópia, e quais problemas isso evita em tempo de execução?

A herança de JS não recebe cópias das propriedades ou métodos. Ao invés disso, ele mantém uma referência ao seu protótipo. Se a propriedade não existe no objeto atual, o JS delega a busca para o `[[Prototype]]`, subindo a cadeia até encontrar ou desistir. Este estilo evita desperdício de memória, dificuldades de manutenção, etc.

2. Explique o que acontece internamente quando você acessa uma propriedade que não existe diretamente em um objeto, detalhando o caminho completo da busca.

Caso uma propriedade não exista diretamente no objeto, o JS inicia um processo automático de busca chamado Delegação.

Primeiro a máquina verifica as propriedades próprias do objeto chamado. Se não existir, ele inicia o `[[prototype]]` que aponta para outro objeto e repete a verificação. Este processo irá se repetir até achar a propriedade desejada. Caso não ache, o objeto retorna null.

3. Qual é a diferença conceitual entre `[[Prototype]]`, `__proto__` e `prototype`, e por que confundir esses três leva a erros difíceis de depurar?

- `[[prototype]]` é o mecanismo interno real da linguagem.
- `__proto__` é uma porta de acesso para o `[[prototype]]`
- `prototype` não pertence a objetos, mas a funções construtoras.

4. Por que métodos definidos no prototype são mais eficientes em memória do que métodos definidos dentro de uma factory function?

Métodos definidos no prototype são mais eficientes em memória porque são instanciados uma única vez e compartilhados por todas as instâncias através da delegação prototipal, evitando a criação de múltiplas cópias da mesma função.

5. Em que situação Object.create é uma escolha melhor do que usar class ou função construtora, e por quê?

O Object.create cria um novo objeto cujo [[Prototype]] aponta para outro objeto. Ele não copia propriedades nem valores.

O que acontece é reutilização via delegação.

6. O que realmente acontece quando um método é sobrescrito em um objeto filho, e como o JavaScript decide qual versão será executada?

Ele não sobrescreve o valor do objeto apontado. A herança não copia valores, após definir um método no objeto filho, o JS cria uma propriedade própria no filho, que oculta a versão existente no protótipo. Durante a execução, o motor sempre utiliza a primeira versão encontrada na cadeia de protótipos, começando no próprio objeto.

```
filho
├─ metodo() ← usado (próprio)
↓
protótipo
├─ metodo() ← existe, mas é ignorado
```

7. Por que Object.defineProperty cria propriedades “travadas” por padrão, e qual o risco de esquecer isso em código real?

Propriedades "travadas" são as que não podem ser alteradas livremente. Deixar elas neste estado, pode até deixar o código funcionando, porém poderão ocorrer erros inesperados futuramente.

8. Qual a diferença prática entre writable: false e configurable: false, e por que elas não resolvem o mesmo problema?

Writable: false - impede de alterar um valor da propriedade, mas não controla diretamente se o valor pode ser alterado.

Configurable: false - impede redefinir ou apagar a propriedade, mas não controla diretamente o valor.

9. Por que Object.freeze não é considerado uma solução completa de imutabilidade em JavaScript?

O Object.freeze só proíbe alterações dentro do próprio objetos. Porém, é possível alterar os mesmos valores fora do objeto

10. Em que cenário o uso de getter e setter pode introduzir bugs difíceis de rastrear, mesmo sendo tecnicamente correto?

Eles dificultam achar erros quando escondem lógica com efeitos colaterais.

- Um getter executa um cálculo pesado ou dependem de estado externo.
- Um setter altera múltiplas propriedades ou dispara ações invisíveis.
- Ler uma propriedade passa a mudar o estado do objeto.

11. Por que `Object.assign` e o operador `spread` não fazem cópia profunda, e qual consequência isso pode causar em sistemas maiores?

Porque tanto `Object.assign` quanto o operador `spread` (`{ ...obj }`) copiam apenas as referências, não os objetos internos. Eles fazem uma cópia rasa: o objeto de fora é novo, mas tudo que está dentro continua apontando para os mesmos valores.

Em sistemas maiores, cópias rasas feitas com `Object.assign` ou `spread` podem causar erros ao alterar objetos internos, pois as referências são compartilhadas entre as cópias, gerando efeitos colaterais inesperados.

12. Explique por que `Map` aceita qualquer tipo como chave, enquanto objetos comuns não foram projetados para isso.

O `Map` aceita qualquer tipo de chave porque foi projetado desde o início para associar valores a como são, não apenas a `Strings`.

Objetos comuns, convertem toda chave para `String`, o que causa colisões e comportamentos anormais.

O `Map` mantém a chave pela referência original, sem conversão, garantindo previsibilidade e evitando conflitos.

13. Em termos de design, quando usar `Map` é mais adequado do que usar um objeto simples como dicionário?

Usamos o `Map` quando as chaves não são `Strings` previsíveis, quando você precisa preservar a identidade do objeto como chave, quando a ordem de inserção importa ou quando há inserções e remoções frequentes.

14. O que é polimorfismo em JavaScript sem herança, e por que ele funciona mesmo sem contratos formais?

Polimorfismo em JavaScript sem herança acontece quando objetos diferentes respondem ao mesmo método ou comportamento, independentemente de compartilharem protótipo ou classe. O que importa não é de onde o método veio, mas se ele existe no momento da chamada. Isso funciona porque JavaScript usa `duck typing`: se o objeto “age como” aquilo que o código espera, ele serve. Não existem contratos formais porque a verificação acontece em tempo de execução, não em tempo de escrita.

15. Por que o JavaScript permite chamar métodos inexistentes sem aviso prévio em tempo de escrita, e qual é o custo dessa decisão?

O JavaScript permite chamar métodos inexistentes sem aviso prévio porque ele é dinâmico e fracamente tipado. O motor só verifica se o método existe no momento da execução, o que dá muita flexibilidade, mas transfere a responsabilidade para o desenvolvedor. O custo dessa decisão é que erros aparecem tarde, como `TypeError: x is not a function`, muitas vezes longe da causa real, tornando bugs mais difíceis de prever, testar e depurar em sistemas grandes.

16. Qual a relação entre `this` e `prototype`, e por que métodos perdem o contexto quando são extraídos do objeto?

A relação entre `this` e `prototype` é indireta.

O `prototype` define onde o método está, e `this` define quem iniciou a chamada.

Quando um método é chamado via objeto, `this` aponta para esse objeto. Quando o método é extraído e chamado isoladamente, ele perde a referência do objeto original, então `this` deixa de apontar para ele, porque o contexto de chamada mudou.

17. Por que `class` em JavaScript é considerado açúcar sintático, e o que realmente muda quando você troca `prototype` por `class`?

`class` é considerado açúcar sintático porque ele não cria um novo modelo de herança. Por baixo dos panos, continua sendo `prototype` e cadeia prototipal. O que muda é apenas a forma de escrever: `class` organiza o código, esconde detalhes e reduz erros comuns, mas não altera o funcionamento interno da linguagem.

18. Em um sistema grande, quais riscos surgem ao modificar diretamente o `prototype` de objetos compartilhados?

19. Explique como `factory functions` combinadas com `prototypes` oferecem um equilíbrio entre controle e reaproveitamento de código.

20. Qual é o principal erro conceitual de quem tenta aplicar princípios de orientação a objetos clássica diretamente em JavaScript?